

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar

PRN: 24070521058

Course: Semester V (B.Tech)

Experiment No. 3

Student Name: Tanmay Sankulwar
PRN: 24070521058
File Path: EXP3/index.html , EXP3/script.js

Experiment Title

SIT NAGPUR | Password Verification (Secure Registration Portal)

Software / Tools Required

1. Visual Studio Code / Antigravity IDE
2. Google Chrome (or modern web browser)
3. HTML5
4. CSS3
5. JavaScript (ES6)

Experiment Program Code

Task 3.a — Password Verification UI with Strength Meter & Validation Checklist

File: EXP3/index.html

The HTML file implements a user registration portal featuring interactive eye toggle buttons for password visibility, a 3-segment color-coded password strength meter, a live 5-point security rule checklist, and confirm-password matching indicator.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SIT NAGPUR | Password Verification</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    body {
      background: linear-gradient(135deg, #0f172a, #1d4ed8, #38bdf8);
      min-height: 100vh;
      display: flex;
      justify-content: center;
```

```
        align-items: center;
        padding: 30px;
    }
    .container {
        width: 500px;
        background: #ffffff;
        border-radius: 20px;
        padding: 35px 40px;
        box-shadow: 0 15px 35px rgba(0, 0, 0, 0.3);
        transition: transform 0.3s ease;
    }
    .container:hover {
        transform: translateY(-2px);
    }
    h1 {
        text-align: center;
        color: #0f172a;
        font-size: 26px;
        margin-bottom: 5px;
    }
    .subtitle {
        text-align: center;
        color: #64748b;
        font-size: 15px;
        margin-bottom: 25px;
    }
    label {
        display: block;
        margin-top: 15px;
        margin-bottom: 8px;
        font-weight: 600;
        color: #1e293b;
        font-size: 14px;
    }
    .input-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }
    input {
        width: 100%;
        padding: 12px 45px 12px 14px;
        border: 1.5px solid #cbd5e1;
        border-radius: 10px;
        font-size: 15px;
        outline: none;
        transition: all 0.3s ease;
        background: #f8fafc;
    }
    input:focus {
        border-color: #2563eb;
        box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.15);
        background: #ffffff;
    }
    .toggle-btn {
        position: absolute;
        right: 14px;
        background: none;
        border: none;
        cursor: pointer;
        color: #64748b;
        display: flex;
        align-items: center;
        justify-content: center;
        padding: 4px;
        border-radius: 50%;
        transition: background-color 0.2s, color 0.2s;
    }
    .toggle-btn svg {
        width: 20px;
        height: 20px;
    }
}
```

```
.strength-meter {
  margin-top: 10px;
}
.strength-bar-container {
  height: 6px;
  background-color: #e2e8f0;
  border-radius: 3px;
  overflow: hidden;
  display: flex;
  gap: 2px;
}
.strength-segment {
  flex: 1;
  height: 100%;
  background-color: transparent;
  transition: background-color 0.3s ease;
}
.strength-text {
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 13px;
  font-weight: 600;
  margin-top: 5px;
  color: #64748b;
}
.rules-list {
  margin-top: 15px;
  list-style: none;
  padding: 0;
}
.rule-item {
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 13.5px;
  color: #64748b;
  margin-bottom: 8px;
  transition: color 0.3s ease;
}
.rule-item.svg {
  width: 16px;
  height: 16px;
  flex-shrink: 0;
  transition: transform 0.2s ease, fill 0.3s ease;
}
.rule-item.valid {
  color: #10b981;
  font-weight: 500;
}
.rule-item.invalid {
  color: #64748b;
}
.feedback-message {
  font-size: 13px;
  margin-top: 6px;
  font-weight: 500;
  display: none;
  align-items: center;
  gap: 6px;
}
.feedback-message.match {
  color: #10b981;
  display: flex;
}
.feedback-message.mismatch {
  color: #ef4444;
  display: flex;
}
button.submit-btn {
  width: 100%;
  padding: 14px;
```

```

        margin-top: 25px;
        background: #2563eb;
        color: white;
        border: none;
        border-radius: 10px;
        font-size: 16px;
        font-weight: bold;
        cursor: pointer;
        transition: all 0.3s ease;
        box-shadow: 0 4px 10px rgba(37, 99, 235, 0.2);
    }
    .alert-box {
        margin-top: 20px;
        padding: 12px 16px;
        border-radius: 10px;
        text-align: center;
        font-size: 14px;
        font-weight: 600;
        display: none;
    }
    .alert-success {
        background-color: #ecfdf5;
        color: #065f46;
        border: 1px solid #a7f3d0;
    }
    .alert-error {
        background-color: #fef2f2;
        color: #991b1b;
        border: 1px solid #fecaca;
    }
    .footer {
        margin-top: 35px;
        text-align: center;
        border-top: 2px solid #f1f5f9;
        padding-top: 20px;
        color: #64748b;
        font-size: 14px;
    }
    .footer b {
        color: #1d4ed8;
    }
</style>
</head>
<body>
<div class="container">
    <h1>Password Verification</h1>
    <p class="subtitle">Secure Registration Portal</p>

    <form id="verificationForm" onsubmit="return handleFormSubmit(event)">
        <label for="username">Enter Username</label>
        <div class="input-wrapper">
            <input type="text" id="username" placeholder="Enter username" required autocomplete="username">
        </div>

        <label for="password">Enter Password</label>
        <div class="input-wrapper">
            <input type="password" id="password" placeholder="Enter strong password" required autocomplete="new-password">

            <button type="button" class="toggle-btn" id="togglePasswordBtn">
                <svg id="eyeOpenIcon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path
ath d="M1 12s4-8 11-8 11 8-4 8-11 8-11-8z"></path><circle cx="12" cy="12" r="3"></circle></svg>
                <svg id="eyeClosedIcon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
style="display:none;"><path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94M
9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19m-6.72-1.07a3 3 0 1 1-4.24-4.24"></path><li
ne x1="1" y1="1" x2="23" y2="23"></line></svg>
            </button>
        </div>

        <div class="strength-meter">
            <div class="strength-bar-container">
                <div class="strength-segment" id="strengthSeg1"></div>
                <div class="strength-segment" id="strengthSeg2"></div>
            </div>
        </div>
    </form>
</div>

```

```

        <div class="strength-segment" id="strengthSeg3"></div>
    </div>
    <div class="strength-text">
        <span>Password Strength:</span>
        <span class="strength-label" id="strengthLabel"><span style="color: #cbd5e1;">Too Weak</span></
span>
    </div>
</div>

<ul class="rules-list">
    <li class="rule-item invalid" id="ruleLength">At least 8 characters long</li>
    <li class="rule-item invalid" id="ruleUppercase">At least one uppercase letter (A-Z)</li>
    <li class="rule-item invalid" id="ruleLowercase">At least one lowercase letter (a-z)</li>
    <li class="rule-item invalid" id="ruleNumber">At least one digit (0-9)</li>
    <li class="rule-item invalid" id="ruleSpecial">At least one special character (@, $, !, %, *, ?,
&)</li>
</ul>

<label for="confirmPassword">Confirm Password</label>
<div class="input-wrapper">
    <input type="password" id="confirmPassword" placeholder="Re-enter password" required autocomplete
="new-password">
    <button type="button" class="toggle-btn" id="toggleConfirmPasswordBtn">
        <svg id="confirmEyeOpenIcon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
="2"><path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z"></path><circle cx="12" cy="12" r="3"></circle></svg>
        <svg id="confirmEyeClosedIcon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
th="2" style="display:none;"><path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.0
6-5.94M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19m-6.72-1.07a3 3 0 1 1-4.24-4.24"></
path><line x1="1" y1="1" x2="23" y2="23"></line></svg>
    </button>
</div>

<div id="confirmFeedback" class="feedback-message"></div>

<button type="submit" class="submit-btn">Register User</button>
</form>

<div id="alertBox" class="alert-box"></div>

<div class="footer">
    <p><b>Developed By</b></p>
    <p><b>Tanmay Sankulwar</b></p>
    <p>PRN : <b>24070521058</b></p>
</div>
</div>
<script src="script.js"></script>
</body>
</html>

```

Task 3.b — Real-time Regex Policy Validation and Password Confirmation

File: EXP3/script.js

The JavaScript code evaluates the password string dynamically using Regular Expressions for character length (`>= 8`), uppercase (`/[A-Z]/`), lowercase (`/[a-z]/`), digit (`/[0-9]/`), and special symbol sets. It adjusts the strength bar dynamically, checks matching between Password and Confirm Password, and manages form submission.

```

document.addEventListener('DOMContentLoaded', function () {
    const passwordInput = document.getElementById('password');
    const confirmPasswordInput = document.getElementById('confirmPassword');
    const togglePasswordBtn = document.getElementById('togglePasswordBtn');
    const toggleConfirmPasswordBtn = document.getElementById('toggleConfirmPasswordBtn');
    const alertBox = document.getElementById('alertBox');

    // Toggle Password Visibility
    togglePasswordBtn.addEventListener('click', function () {

```



```
setRuleState('ruleLowercase', hasLower);
setRuleState('ruleNumber', hasDigit);
setRuleState('ruleSpecial', hasSpecial);

let score = 0;
if (hasLength) score++;
if (hasUpper) score++;
if (hasLower) score++;
if (hasDigit) score++;
if (hasSpecial) score++;

updateStrengthMeter(score);
return score === 5;
}

function updateStrengthMeter(score) {
  const seg1 = document.getElementById('strengthSeg1');
  const seg2 = document.getElementById('strengthSeg2');
  const seg3 = document.getElementById('strengthSeg3');
  const label = document.getElementById('strengthLabel');

  seg1.style.backgroundColor = '';
  seg2.style.backgroundColor = '';
  seg3.style.backgroundColor = '';

  if (score === 0) {
    label.innerHTML = '<span style="color: #cbd5e1;">Too Weak</span>';
  } else if (score <= 2) {
    seg1.style.backgroundColor = '#ef4444'; // Red
    label.innerHTML = '<span style="color: #ef4444;">Weak</span>';
  } else if (score <= 4) {
    seg1.style.backgroundColor = '#f59e0b'; // Orange
    seg2.style.backgroundColor = '#f59e0b';
    label.innerHTML = '<span style="color: #f59e0b;">Medium</span>';
  } else if (score === 5) {
    seg1.style.backgroundColor = '#10b981'; // Green
    seg2.style.backgroundColor = '#10b981';
    seg3.style.backgroundColor = '#10b981';
    label.innerHTML = '<span style="color: #10b981;">Strong</span>';
  }
}

function checkPasswordMatch() {
  const pwd = passwordInput.value;
  const cpwd = confirmPasswordInput.value;
  const feedback = document.getElementById('confirmFeedback');

  if (cpwd === "") {
    feedback.style.display = "none";
    feedback.className = "feedback-message";
    return false;
  }

  feedback.style.display = "flex";
  if (pwd === cpwd) {
    feedback.className = "feedback-message match";
    feedback.innerHTML = `Passwords match`;
    return true;
  } else {
    feedback.className = "feedback-message mismatch";
    feedback.innerHTML = `Passwords do not match`;
    return false;
  }
}

window.handleFormSubmit = function (event) {
  event.preventDefault();
  const username = document.getElementById('username').value.trim();
  const isValid = validatePassword();
  const isMatch = checkPasswordMatch();

  alertBox.style.display = 'none';
}
```

```
    if (!isPasswordValid) {
        showAlert('Password does not meet all security requirements.', 'error');
        return false;
    }
    if (!isMatch) {
        showAlert('Passwords do not match.', 'error');
        return false;
    }

    showAlert(`Registration successful! Account created for <strong>${username}</strong>`, 'success');
    return true;
};

function showAlert(message, type) {
    alertBox.innerHTML = message;
    alertBox.style.display = 'block';
    alertBox.className = type === 'success' ? 'alert-box alert-success' : 'alert-box alert-error';
}
});
```

Output

1. **Real-time Checklist Feedback:** As the user types into the password input, each rule item instantly shifts from gray (unmet) to green checkmarks (met) using regex evaluation.
2. **Dynamic Strength Meter:** - **Score 1–2:** Red single bar (Weak). - **Score 3–4:** Amber dual bars (Medium). - **Score 5:** Green triple bars (Strong).
3. **Password Matching Feedback:** Live feedback displays "Passwords match" in emerald green or "Passwords do not match" in red below the confirmation input.
4. **Form Submission Alert:** Submitting an incomplete form shows an error banner, whereas meeting all 5 security rules and confirming matching credentials displays "Registration successful! Account created for <username>." .

Screenshot

Validation Checklist & Password Verification Interface

Password Verification
Secure Registration Portal

Enter Username

Enter Password

Password Strength: Too Weak

- 1 At least 8 characters long
- 2 At least one uppercase letter (A-Z)
- 3 At least one lowercase letter (a-z)
- 4 At least one digit (0-9)
- 5 At least one special character (@, \$, %, *, ?, &)

Confirm Password

Register User

Developed By
Tanmay Sankulwar
PRN : **24070521058**

Password Verification Interface

Password Validation & Match Confirmation State

Palindrome Checker

Enter a Word:

Check

Password Verification Result

Result / Conclusion

Experiment 3 demonstrated advanced client-side form validation techniques using JavaScript Regular Expressions (RegExp). Key programming concepts implemented include pattern testing (`regex.test()`), dynamic CSS class toggling, SVG DOM manipulation, multi-stage password strength scoring algorithms, and event-driven form lifecycle handling.